



Testeo Automatizado

TESTEO AUTOMATIZADO

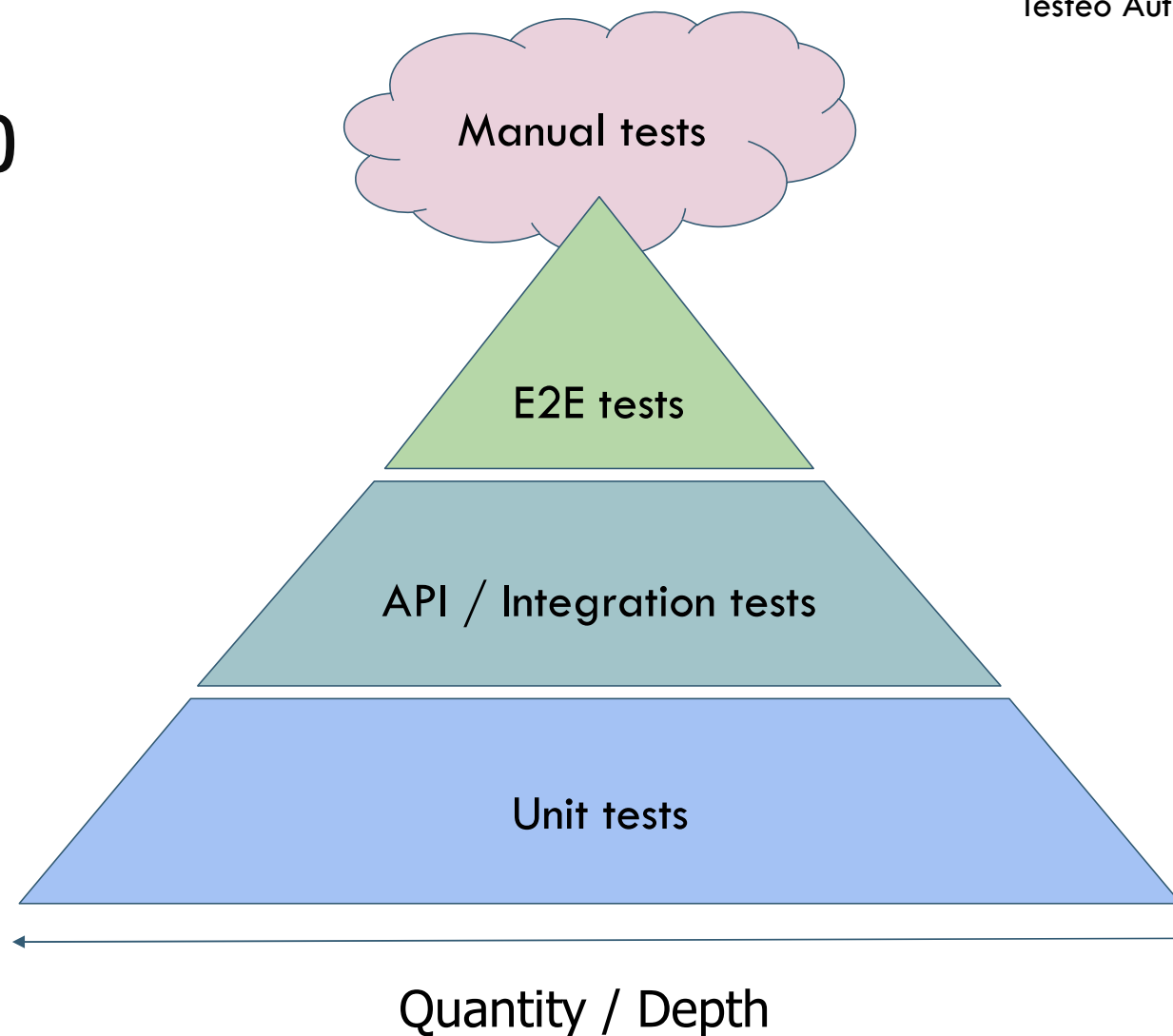
Unit Testing



TESTING PYRAMID

Slower execution
Coverage +
More difficult maintenance
More brittle
Longer to write

Faster execution
Coverage –
Easier maintenance
More robust
Quicker to write





*“El costo de arreglar un defecto
descubierto en la fase de diseño es 100
veces menor que el costo de arreglarlo
después de que el software ha sido
lanzado”*

Barry Boehm





SETUP CODE COVERAGE FOR YOUR BACK END

- Investigate how to setup **nyc** tool for code coverage with Mocha JS
- Create a few examples of functions and unit tests
- Compute coverage and generate report





SINON.JS

MOCKING CONCEPTS

System in Production



- Component Under Test
- Depended on Components
- Additional Components

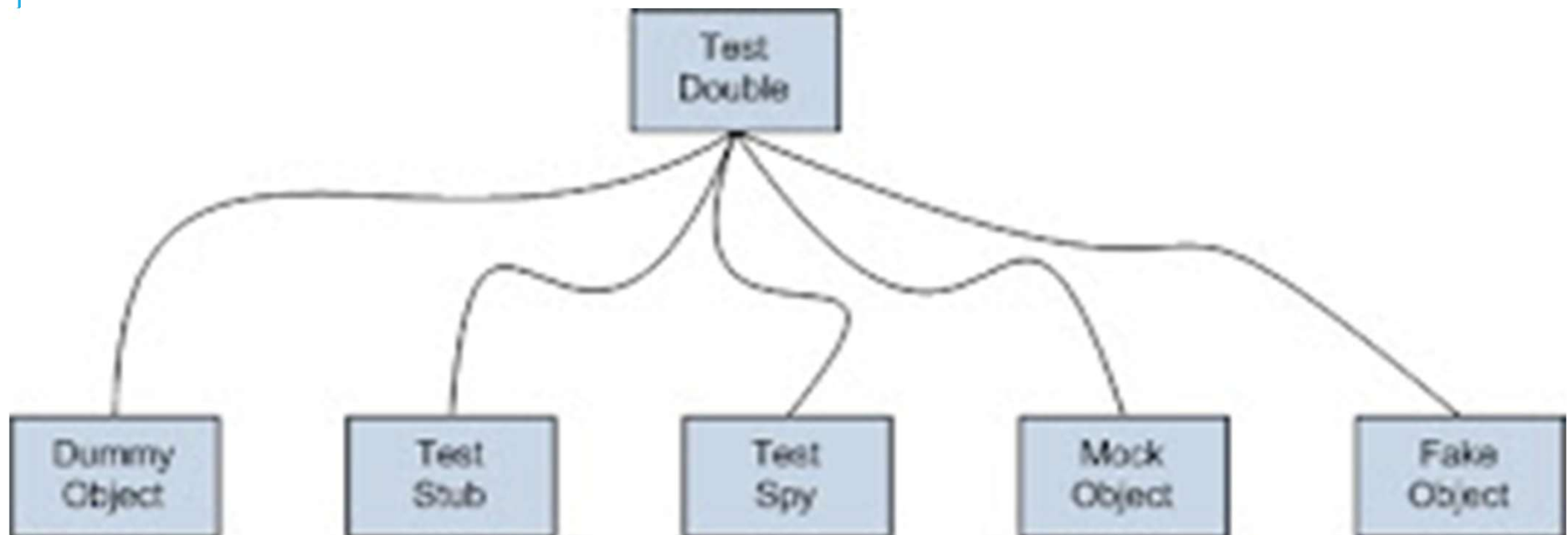
System in Unit Test



- Component Under Test
- Mocks for Components



TEST DOUBLE





SPIES

Permiten que la entidad duplicada conserve su comportamiento original al tiempo que proporciona información sobre cómo interactuó con SUT.

*El **spy** puede decirle a la prueba qué parámetros se le dieron, cuántas veces se llamó y cuál fue el valor de retorno, si lo hubo.*

Una forma de esto podría ser un servicio de correo electrónico que registre cuántos mensajes se enviaron.



STUBS

*La función del **stub** es devolver valores controlados al objeto que se está probando. Estos se describen como entradas indirectas a la prueba.*

Generan entradas predeterminadas para el SUT.

Permite ejecutar rutas de código no probadas.

Hay dos tipos:

Respondedor: inyecta valores válidos

Saboteador: inyecta errores o excepciones



STUBS

*Los **stubs** son como espías, con la diferencia de que reemplazan la función objetivo. También pueden contener comportamientos personalizados, como devolver valores o lanzar excepciones. Incluso pueden llamar automáticamente a cualquier función de devolución de llamada proporcionada como parámetro.*



STUBS

*Los **stubs** tienen varios usos comunes:*

- *Puedes usarlos para reemplazar fragmentos de código problemáticos.*
- *Puedes usarlos para activar rutas de código que de otro modo no se activarían, como el manejo de errores.*
- *Puedes usarlos para facilitar las pruebas de código asíncrono.*



MOCKS

*Los objetos **mocks** se utilizan para verificar el comportamiento de los objetos durante una prueba.*

Permiten verificar la interacción con una dependencia simulada, como la cantidad de veces que se llama a un método o los parámetros que se le pasan.



EN RESUMEN SU USO:

Stubs

Cuando necesitas respuestas predefinidas para un componente sin verificar su interacción.

Mocks

Cuando necesitas verificar la interacción con una dependencia simulada (por ejemplo, cuántas veces se llamó a un método).

Spies

Cuando quieres verificar la interacción con una dependencia simulada y también llamar a las implementaciones reales de los métodos.



FAKES

Estos objetos tienen sus implementaciones, pero generalmente toman algún atajo que los hace no aptos para producción (ej. una base de datos en memoria, una capa de servicio falsa).

Reemplazan parcial o completamente la dependencia.

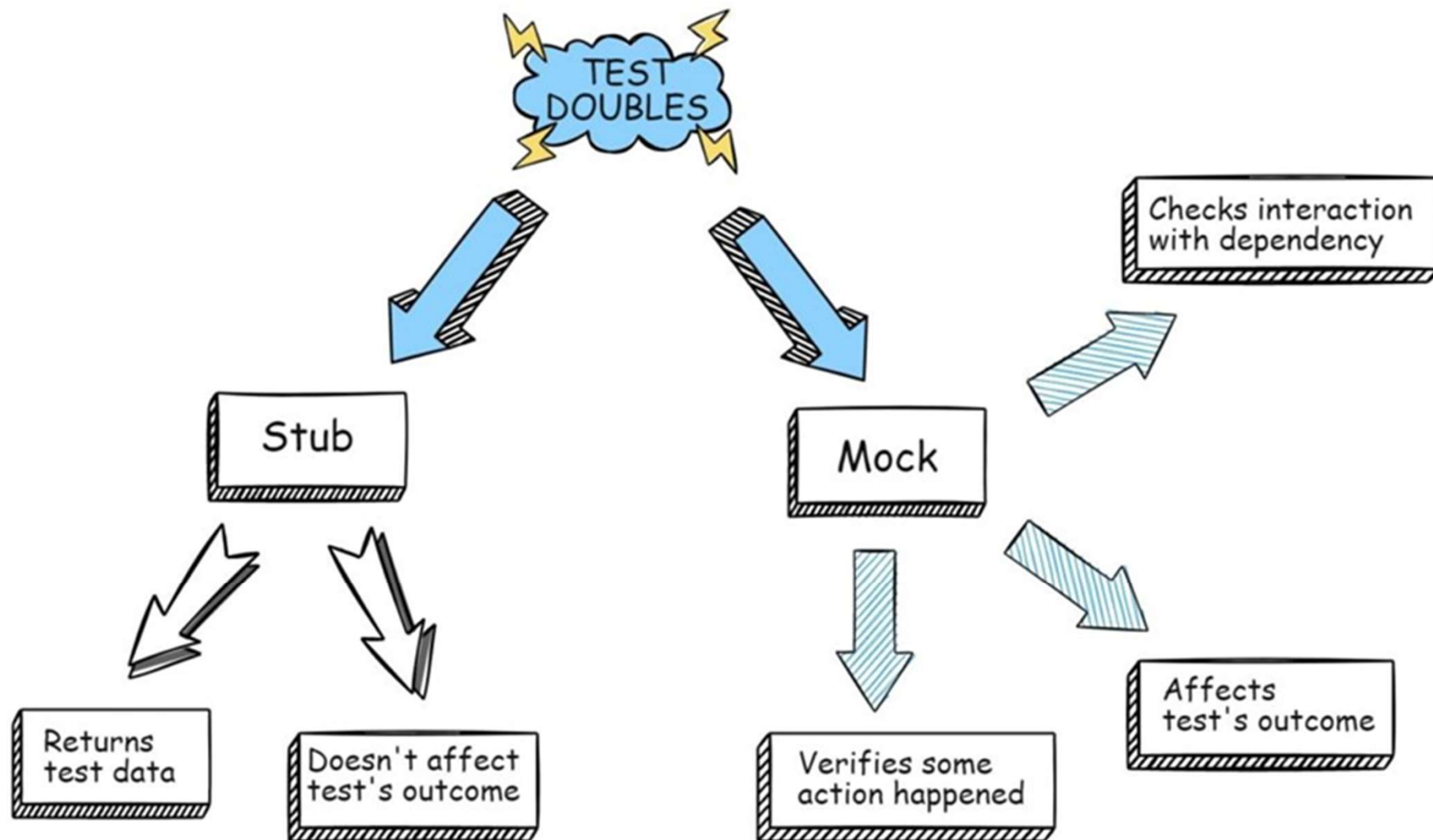
No es controlado ni observado por las pruebas.



DUMMY

Estos objetos se pasan como argumento pero nunca se usan realmente. Por lo general, solo se usan para completar listas de parámetros.

Su propósito es estar ahí cuando la sintaxis lo requiera.





SINON



- *Sinon is JavaScript framework to create test spies, stubs and mocks that can be used with any JavaScript unit testing framework.*
- *npm install sinon*
- *var sinon = require("sinon");*



Testeo Automatizado

MUCHAS GRACIAS